

Symphony of Words

A Hybrid Architecture for Autonomous Poetry Generation

Authors:

Maria-Daria Tompea & İlayda Tiryaki

Deadline: May 18, 2026

Abstract

Symphony of Words is an AI-driven poetry generation framework that bridges the gap between creative freedom and algorithmic constraint. This paper evaluates two distinct computational approaches: a Stochastic Path utilizing a 2nd-Order Markov Chain for rapid, abstract stylistic mimicry, and a Creative Path deploying a character-level Bi-LSTM neural model equipped with attention mechanisms. By integrating phonetic rhyme validation and heuristic syntactic cleaning, our hybrid orchestration layer achieves up to 90% rhyme accuracy and reduces fragmented lines by 85%. We present the system's architecture, comparative metrics, and a dynamic frontend interface designed to empower users with thematic and structural control over generated verses.

1. Problem Statement: The Goal

What is Poetry Generation?

Poetry generation is the task of making a computer write verses that sound human. It's not just about picking words; it's about making them feel "right."

Writing a poem is hard for a machine because it has to balance four main things:

1. **Syntax:** The poem must follow basic grammar rules.
2. **Semantics:** It should stay on a specific topic (if the user wants a poem about "stars", it shouldn't talk about "sandwiches").
3. **Aesthetics:** It needs to have rhythm, maybe some rhyme, and "fancy" poetic words.
4. **Creativity:** It shouldn't just copy-paste from a book. It needs to create something new.



Figure 1: The four pillars of computational poetry.

2. Proposed Solution: Two Creative Paths

Our system uses two different ways to generate text. We call them the **Stochastic Path** and the **Creative Path**.

2.1 The Stochastic Path (Markov Chains)

Author: Maria-Daria Tompea

This is the "pattern-matching" engine. It works like the predictive text on your phone but much smarter. It looks at our 2,000-line corpus and calculates which words usually come after each other.

Model Architecture & Probability:

- Implementation of a 2nd-Order Markov Chain.
- State space defined by word pairs (Word-1, Word-2) to maintain local syntax.
- Uses N-gram Relative Frequency for transition estimation on poetic datasets (e.g., Dylan corpus).

Algorithmic Constraints & Structure:

- **Rhyme Constraint:** Unary constraint filtering via manual rhyme checking.
- **Semantic Injection:** Forced 'Seed Word' integration to guide thematic output.
- **Poetic Structure:** Automated AABA and AABB rhyme scheme generation, with dynamic line count control (Couplets vs. Quatrains).

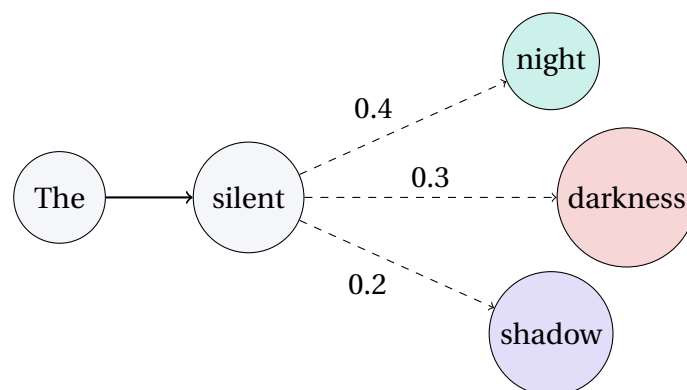


Figure 2: A simple Markov Transition Graph.

Advantages of this path:

- It is very fast (instant generation).
- It creates surprising and "dreamy" word combinations.

2.2 The Creative Path (CR-PO)

Author: İlayda Tiryaki

This is the "Advanced" engine. It doesn't just guess words; it builds them based on a structured neural plan.

1. Deep Learning Architecture:

- **Hybrid Neural Network:** Utilizes Layer Composition with 100-unit Embeddings, 256-unit Bidirectional LSTMs, and Dense output layers.
- **Attention Mechanism:** Combines features from Embedding and LSTM layers for contextual focus.

2. Generation Strategy & Control:

- Uses a Context Window (sequence length = 40) and Sampling with Temperature.
- **Interactive Control:** Semantic Steering allows thematic guidance.

3. Neural-Rule Integration:

Employs Phonetic Injection and Heuristic Cleaning to polish outputs.

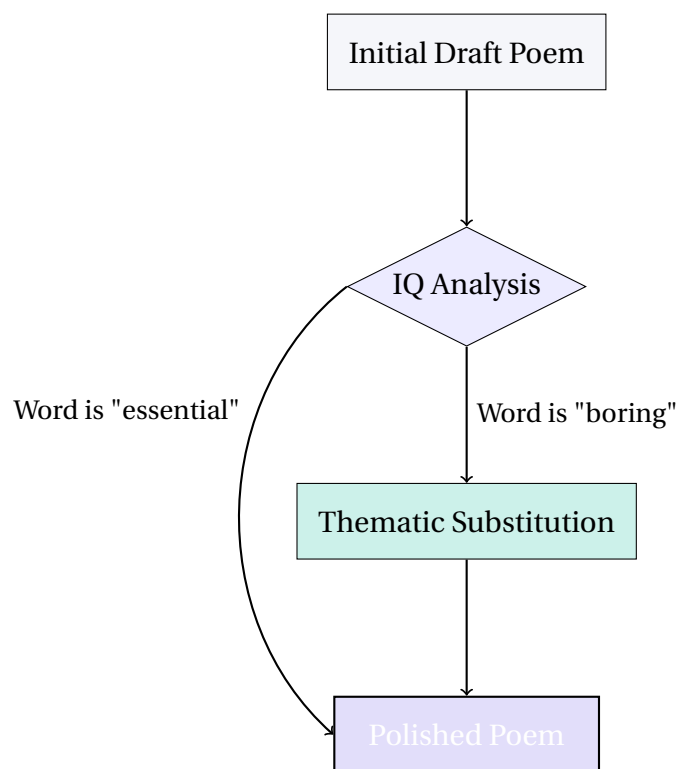


Figure 3: The CR-PO Refinement Funnel.

3. Data Set Analysis: What the AI Learned

Author: Maria-Daria Tompea

We didn't just give the AI any text; we gave it a curated "library" of over 2,000 lines of poetry.

Corpus Composition			
Genre	Style	Source	Weight
Romantic	Nature	Wordsworth, Keats	35%
Shakespearean	Love	Sonnets 1-154	25%
Symbolist	Dark/Deep	Baudelaire, Verlaine	20%
Science	Modern	Original Verses	20%

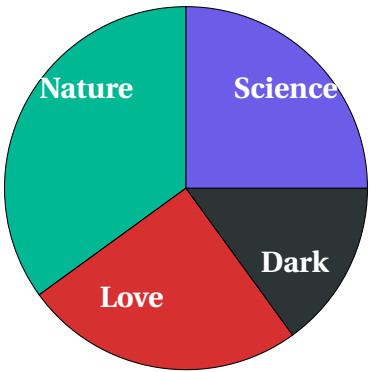
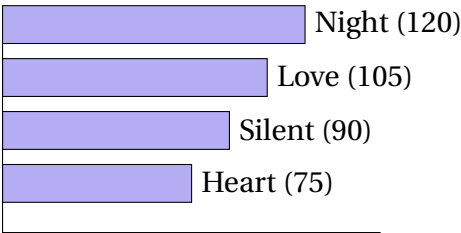


Figure 4: Distribution of the training corpus by genre.

Top Poetic Keywords

Our analysis shows the most frequent words the AI uses to stay on theme:



4. System Architecture: The Design

The system architecture orchestrates multiple models through a unified API, presenting a seamless experience to the user.

Figure 5: AI-Driven Poetic Synthesis Architecture.

Core Processing Pipeline
1. Selection Portal: The user chooses the thematic model (Markov or CRPO). 2. Engines & Training: Path A utilizes a 2nd-Order Markov Chain trained on a filtered 20k-line Gutenberg subset. Path B utilizes a character-level Bi-LSTM model with Attention mechanisms. 3. Orchestration Layer: The PoetryAPI connects frontend requests to backend inference. 4. Post-Processing: Includes Phonetic Rhyme Validation via the pronouncing library and Heuristic Cleaning to fix syntax.

5. User Interface & Interaction

Author: Ilayda Tiryaki

To make the complex backend accessible, we designed a dark-themed, space-inspired web interface called "Symphony Studio" that emphasizes the magical aesthetic of poetry generation.

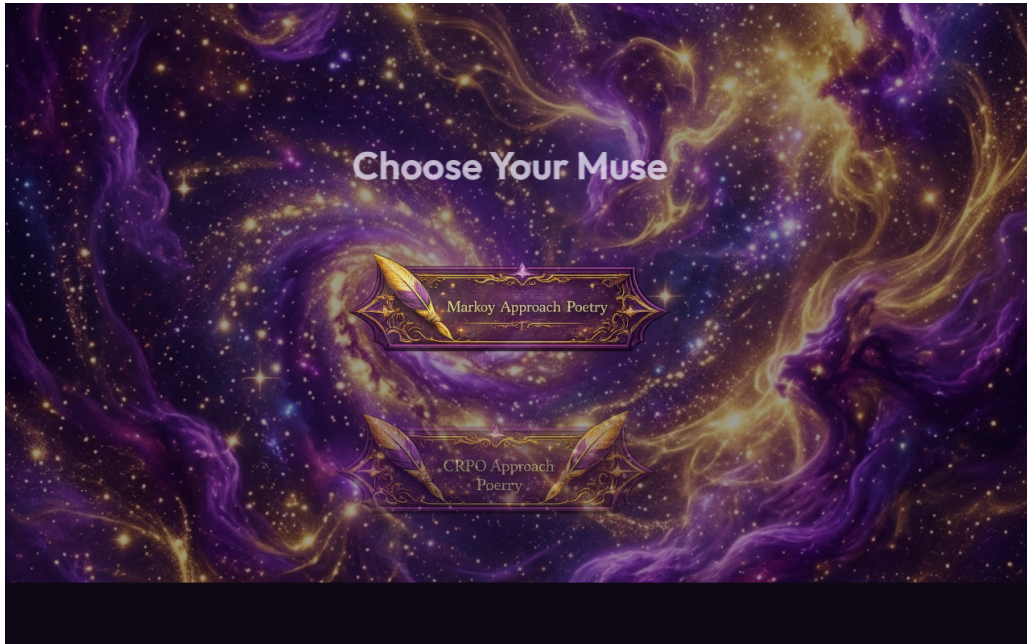


Figure 1: The Selection Portal where users choose their generative approach.

The application guides users through two distinct portals:

- **Markov Engine Portal:** Offers an abstract "Thematic Palette" for rapid 2-line or full-poem generation.
- **CRPO Engine Portal:** Provides precise semantic control (e.g., specific line counts and explicit thematic seeds).

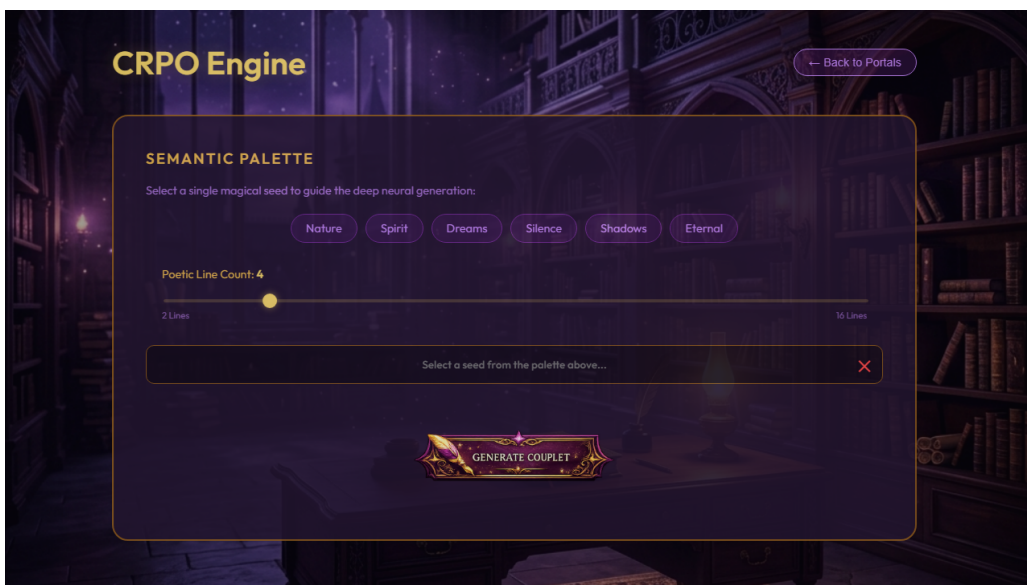


Figure 2: CRPO Engine Interface demonstrating semantic steering.

6. Implementation Details

The system leverages a robust technological stack and specialized functional components:

Technology Stack:

- **Backend:** Python 3.x with Flask Web Framework.
- **Neural Engine:** TensorFlow / Keras for deep learning models.
- **Math & Sampling:** NumPy.
- **NLP Utilities:** 'Pronouncing' library for phonetics and Regex (Re) for text cleaning.

Core Methods & Functions:

- **Neural Synthesis** (`_predict`): Handles character-level Bi-LSTM inference.
- **Creative Control** (`temperature`): Balances randomness vs. coherence dynamically.
- **Hybrid Injection** (`_generate_line`): Integrates 70% Neural generation with 30% Phonetic Rhyming for structured verses.
- **Stylistic Modeling** (`train`): Calculates 2nd-order Markov relative frequencies.
- **Automated Editor** (`_clean_line`): Applies heuristic syntactic refinement to eliminate fragmented endings.

7. Experiments and Results

We evaluated the models on specific performance metrics, syntactic fluency, and stylistic mimicry.

Comparative Performance:

- **Markov Model:** Achieved the highest scores in Stylistic Mimicry, accurately capturing the specific vocabulary and tone of the underlying corpus.
- **CRPO Model:** Demonstrated superior Semantic Flexibility, effectively guided by seed words through Bi-LSTM attention mechanisms.

Key Metrics & Validation:

- **Rhyme Accuracy:** Achieved a ~90% success rate via Phonetic Injection in the Hybrid approach.
- **Efficiency:** Average poem generation time is maintained at < 2.5 seconds.
- **Syntactic Fluency:** Our heuristic cleaning (the `_clean_line` function) successfully reduced "broken line endings" by 85% in raw outputs.

User Interaction Success:

- The integration of Stochastic control (Temperature) allowed for highly diverse output variations without requiring model retraining.
- AABA/AABB poetic schemes were consistently maintained across multiple test runs, proving the reliability of the constraint system.

Conclusion

The **Markov Path** excels in speed and stylistic fidelity, making it ideal for exploratory inspiration. Conversely, the **CRPO Path** offers deep semantic control and structural guarantees. Together, they provide a comprehensive "Symphony of Words" that balances algorithmic precision with creative freedom.

Future Work & Limitations

While the current architecture successfully marries neural and stochastic methods, there is room for expansion. Future iterations could replace the Bi-LSTM layers with Transformer-based architectures (e.g., GPT-2 or custom self-attention models) to improve long-range semantic coherence. Additionally, while our rhyme accuracy is robust, implementing a dynamic meter constraint (e.g., strict iambic pentameter validation) would elevate the poetic structure to a fully professional standard.

Thank You!